

1. 自己改良的 print_cfg.py :

Import:

```
1  import argparse
2  import sys
3  import yaml
4  import os
5  from typing import Dict, Any, Optional
```

load_yaml_config:

```
8  def load_yaml_config(file_path: str) -> Dict[str, Any]:
9      """載入 YAML 配置文件並返回解析後的字典。
10
11      Args:
12          file_path: YAML 文件路徑
13
14      Returns:
15          解析後的配置字典
16
17      Raises:
18          SystemExit: 如果文件不存在或無法解析
19      """
20      if not os.path.isfile(file_path):
21          print(f"[錯誤] YAML 文件不存在: {file_path}", file=sys.stderr)
22          sys.exit(1)
23
24      try:
25          with open(file_path, "r", encoding="utf-8") as f:
26              cfg = yaml.safe_load(f)
27      except yaml.YAMLError as e:
28          print(f"[錯誤] YAML 解析失敗: {e}", file=sys.stderr)
29          sys.exit(1)
30      except Exception as e:
31          print(f"[錯誤] 讀取文件時發生錯誤: {e}", file=sys.stderr)
32          sys.exit(1)
33
34      if cfg is None:
35          print("[錯誤] YAML 文件為空或格式不正確", file=sys.stderr)
36          sys.exit(1)
37
38      return cfg
```

print_block_list:

```
41 def print_block_list(cfg: Dict[str, Any]) -> None:
42     """列印所有頂層配置區塊名稱。
43
44     Args:
45         cfg: 配置字典
46     """
47     print("可用的配置區塊:")
48     for key in sorted(cfg.keys()):
49         print(f"    - {key}")
50
```

print_block_content:

```
52 def print_block_content(block: Any, block_name: str, parameter: Optional[str] = None) -> None:
53     """列印配置區塊的內容。
54
55     Args:
56         block: 要列印的配置區塊
57         block_name: 區塊名稱
58         parameter: 要列印的特定參數名稱 (None 表示列印全部)
59     """
60     print(f"\n{'=' * 50}")
61     print(f"    {block_name.upper()} 區塊")
62     print(f"{'=' * 50}")
63
64     if parameter is None or parameter == "all":
65         # 列印所有參數
66         if isinstance(block, dict):
67             for key, value in sorted(block.items()):
68                 print(f"{key:20s}: {value}")
69         elif isinstance(block, list):
70             for i, item in enumerate(block):
71                 print(f"[{i}]: {item}")
72         else:
73             print(block)
74     else:
75         # 列印特定參數
76         if isinstance(block, dict):
77             if parameter in block:
78                 print(f"{parameter:20s}: {block[parameter]}")
79             else:
80                 print(f"[錯誤] 參數 '{parameter}' 不存在於區塊 '{block_name}' 中。", file=sys.stderr)
81                 sys.exit(1)
82         else:
83             print(f"[錯誤] 區塊 '{block_name}' 不是字典類型，無法查詢參數。", file=sys.stderr)
84             sys.exit(1)
85
86     print()
87
```

Main(前半):

```
89 def main() -> None:
90     """主函數：解析命令行參數並執行相應操作。"""
91     parser = argparse.ArgumentParser(
92         description="從 YAML 配置文件中列印特定的區塊或參數。",
93         formatter_class=argparse.RawDescriptionHelpFormatter
94     )
95
96     parser.add_argument(
97         "--opt",
98         type=str,
99         required=True,
100         help="YAML 配置文件路徑"
101     )
102
103     parser.add_argument(
104         "--stage",
105         type=str,
106         default=None,
107         help="選擇要列印的配置區塊 (例如：train, val, network_g, optim_g)"
108     )
109
110     parser.add_argument(
111         "--parameter",
112         type=str,
113         default="all",
114         help="要列印的參數名稱 (預設：all，表示列印所有參數)"
115     )
116
117     parser.add_argument(
118         "--h",
119         action="store_true",
120         help="列出所有頂層配置區塊並退出"
121     )
122
123     args = parser.parse_args()
```

Main(後半):

```
124
125     # 載入 YAML 配置
126     cfg = load_yaml_config(args.opt)
127
128     # 顯示區塊列表
129     if args.h:
130         print_block_list(cfg)
131         return
132
133     # 必須指定 stage (除非使用 --h)
134     if args.stage is None:
135         print("[錯誤] 請指定 --stage <區塊名稱>", file=sys.stderr)
136         print("\n使用 --h 查看可用的區塊列表。")
137         sys.exit(1)
138
139     block_name = args.stage
140
141     # 檢查區塊是否存在
142     if block_name not in cfg:
143         print(f"[錯誤] 區塊 '{block_name}' 不存在於 YAML 文件中。", file=sys.stderr)
144         print("\n可用的區塊:")
145         for key in sorted(cfg.keys()):
146             print(f"    - {key}")
147         sys.exit(1)
148
149     block = cfg[block_name]
150
151     # 列印區塊內容
152     param = args.parameter if args.parameter != "all" else None
153     print_block_content(block, block_name, param)
154
155
156 if __name__ == "__main__":
157     main()
158
```

2. Python 執行環境建置與使用過程：

以前寫過 discord.py，所以有 python 的環境。

最初在執行

```
python print_cfg.py --opt Retinexformer_LOL_v1.yml --stage train
```

這段指令時持續遭遇

ModuleNotFoundError: No module named 'yaml'

的問題，經過查詢網路發現，要讓 python 使用 yaml 的 module 需要先使用

Pip install pyyaml (有安裝成功!!!!)

但接下來就奇怪了

即便我已經安裝成功 pyyaml，仍然無法使用 import yaml

首先懷疑是 pip 版本問題，於是執行

```
python.exe -m pip install --upgrade pip
```

terminal 回傳

```
C:\msys64\ucrt64\bin\python.exe: No module named pip
```

發現怪異的点，為何路徑是”msys64/ucrt64/bin/python.exe”

這表示 terminal 在運行 print_cfg.py 時所連結到的編譯是透過 msys64 安裝

經過查詢發現在透過 msys64 安裝 MinGW 時所附帶的工具 python。

因為 msys64 在環境變數中的順序相較 python 靠前，搶佔了原本 python 的

路徑，又因為 msys64 本身有 pacman 的用法，所以其附帶的閹割版

python 無法使用 pip，至此所有的原因就明瞭了。

所以可以使用 Windows Python 啟動器。

也就是將

```
python print_cfg.py --opt Retinexformer_LOL_v1.yml --stage train
```

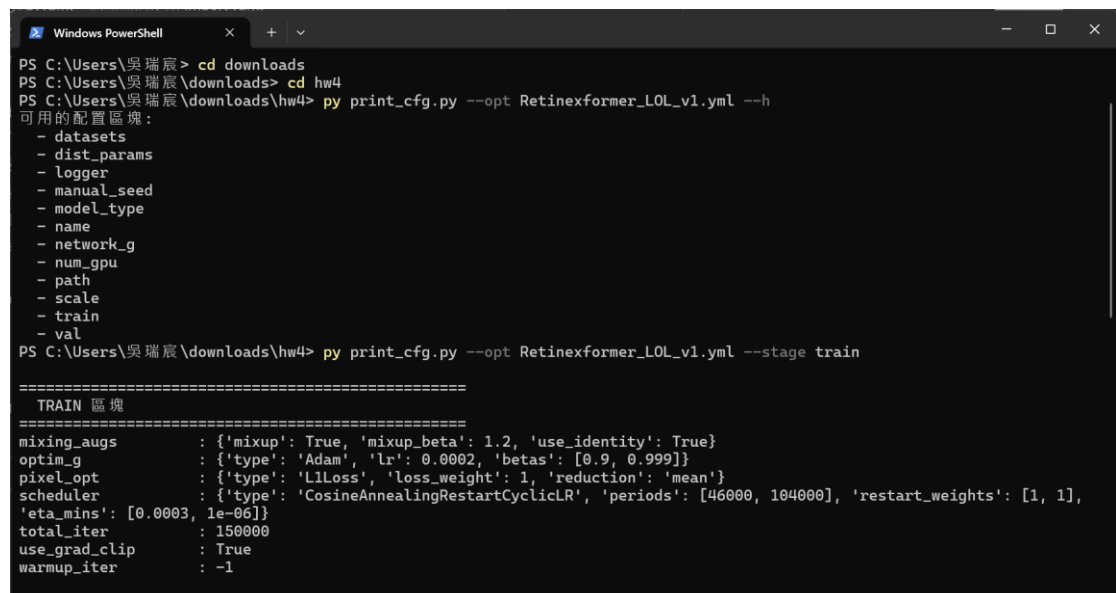
或

```
python.exe print_cfg.py --opt Retinexformer_LOL_v1.yml --stage train
```

改為使用

```
py print_cfg.py --opt Retinexformer_LOL_v1.yml --stage train
```

便可以成功執行，以下：



```
Windows PowerShell
PS C:\Users\吳瑞宸> cd downloads
PS C:\Users\吳瑞宸\downloads> cd hw4
PS C:\Users\吳瑞宸\downloads\hw4> py print_cfg.py --opt Retinexformer_LOL_v1.yml --h
可用的配置區塊:
- datasets
- dist_params
- logger
- manual_seed
- model_type
- name
- network_g
- num_gpu
- path
- scale
- train
- val
PS C:\Users\吳瑞宸\downloads\hw4> py print_cfg.py --opt Retinexformer_LOL_v1.yml --stage train

=====
TRAIN 區塊
=====
mixup_augs      : {'mixup': True, 'mixup_beta': 1.2, 'use_identity': True}
optim_g         : {'type': 'Adam', 'lr': 0.0002, 'betas': [0.9, 0.999]}
pixel_opt       : {'type': 'L1Loss', 'loss_weight': 1, 'reduction': 'mean'}
scheduler       : {'type': 'CosineAnnealingRestartCyclicLR', 'periods': [46000, 104000], 'restart_weights': [1, 1],
'eta_mins': [0.0003, 1e-06]}
total_iter      : 150000
use_grad_clip   : True
warmup_iter     : -1
```

```
Windows PowerShell
PS C:\Users\吳瑞宸\downloads\hw4> py print_cfg.py --opt Retinexformer_L0L_v1.yml --stage train --parameter total_iter

=====
TRAIN 區塊
=====
total_iter      : 150000

PS C:\Users\吳瑞宸\downloads\hw4> py print_cfg.py --opt Retinexformer_L0L_v1.yml --stage network_g

=====
NETWORK_G 區塊
=====
in_channels     : 3
n_feat          : 60
num_blocks      : [1, 2, 2]
out_channels    : 3
stage           : 1
type            : RetinexFormer

PS C:\Users\吳瑞宸\downloads\hw4> py print_cfg.py --opt Retinexformer_L0L_v1.yml --stage optim_g --parameter lr
[錯誤] 區塊 'optim_g' 不存在於 YAML 文件中。

可用的區塊：
- datasets
- dist_params
- logger
- manual_seed
- model_type
- name
```

```
Windows PowerShell
PS C:\Users\吳瑞宸\downloads\hw4> py print_cfg.py --opt Retinexformer_L0L_v1.yml --stage network_g

=====
NETWORK_G 區塊
=====
in_channels     : 3
n_feat          : 60
num_blocks      : [1, 2, 2]
out_channels    : 3
stage           : 1
type            : RetinexFormer

PS C:\Users\吳瑞宸\downloads\hw4> py print_cfg.py --opt Retinexformer_L0L_v1.yml --stage optim_g --parameter lr
[錯誤] 區塊 'optim_g' 不存在於 YAML 文件中。

可用的區塊：
- datasets
- dist_params
- logger
- manual_seed
- model_type
- name
- network_g
- num_gpu
- path
- scale
- train
- val

PS C:\Users\吳瑞宸\downloads\hw4> |
```